Downloading the base ARM system

First, you need to install *debootstrap* for downloading the Debian ARM system, as follows:

```
$ apt-get install debootstrap
```

Now download the minimal Debian ARM system. You will need root permission to download the system using *debootstrap*.

```
$ qemu-debootstrap --arch armel stretch armDebian http://deb.
debian.org/debian/
```

Here, *armDebian* is the directory inside which you need to install the system and *stretch* is the Debian flavour you have to use. It may take a little time to download and configure the system with the above command but once complete, you can start chrooting inside it.

Starting chroot

Before chrooting inside *armDebian* you need to mount a few things inside that environment to make it work properly. It's better to put those commands on a script so that every time you want to chroot, you can just run the script.

```
#!/bin/bash
mount --bind /dev/ /home/0day/armDebian/dev
mount --bind /dev/pts /home/0day/armDebian/dev/pts
mount --bind /dev/shm /home/0day/armDebian/dev/shm
mount -t sysfs sysfs /home/0day/armDebian/sys
mount -t proc proc /home/0day/armDebian/proc
cp -L /etc/resolv.conf /home/0day/armDebian/etc/resolv.conf
chroot /home/0day/armDebian
```

Once you have created this script, just change its permissions and run it using the following commands:

```
$ chmod +x runArm.sh
$ sudo ./runArm.sh
```

Here, *runArm.sh* is the name of the script. Now you are inside a minimal ARM based system. The first thing that you should do inside the system is to update and install the few required packages like *gdb*, Python, etc.

ARM systems don't require BIOS for their functionality since the bootloader is directly installed on hardware. They have U-boot installed as their bootloader. If you want to play around with U-boot or want a full ARM environment, then you cannot do that in chroot. For that, you need to install it inside a full virtual environment like QEMU to get the booting process. We will look into how to do that using the Linaro toolchain next.

Setting up an ARM VM using Linaro

We can use the Linaro tool to set up an ARM VM and save ourselves from chroot and cross-compilation work. For this method, too, you need to install QEMU and the other required packages, as follows:

```
$sudo apt-get install qemu qemu-system qemu-system-arm
```

Add Linaro's repository, containing its tools and a more recent version of QEMU. After that, install the Linaro image-tools, as follows:

```
$ sudo add apt-repository ppa:linaro-maintainers/tools
$ sudo apt-get install linaro-image-tools
```

For cross-compile support, again use the following command:

```
$ sudo apt-get install gcc-arm-linux-gnueabi g++-arm-linux-
gnueabi
```

Downloading and configuring the packages

Now, let's download the Linaro release and hardware pack to run the VM:

```
$ wget https://releases.linaro.org/platform/linaro-n/nano/
alpha-3/linaro-natty-nano-tar-20110302-0.tar.gz

$ wget https://releases.linaro.org/platform/linaro-n/hwpacks/
alpha-3/hwpack_linaro-vexpress_20110302-0_armel_supported.
tar.gz
Create the image using previously instlled linaro-image-tools
$ linaro-media-create --rootfs ext2 --image_file vexpress.img
--dev vexpress \
  --binary linaro-natty-nano-tar-20110302-0.tar.gz \
  --hwpack hwpack_linaro-vexpress_20110302-0_armel_supported.
tar.gz
```

Booting up the VM

Now extract the kernel and *initrd* using the following command (you can also do that manually):

```
$ (IMG=vexpress.img ; if [ -e "$IMG" ] ; then sudo mount -o
loop,offset="$(file "$IMG" | awk 'BEGIN { RS=";"; } /partition
2/ { print $7*512; }')" -t auto "$IMG" /mnt/mnt; else echo
"$IMG not found"; fi )
```

Copy *boot* for booting the VM:

```
$ sudo cp -vr /mnt/mnt/boot .
$ sudo chown -R youruser:youruser boot
$ sudo umount /mnt/mnt
```

You will end up with a number of images in *.boot*:

```
abi-3.3.0-1800-linaro-lt-vexpress-a9
```